

NCERT Solutions For Class 12 Computer Science

Chapter 4 – Queue

1. Fill in the blank

_____ is a linear list of elements in which insertion and deletion takes place from different ends.

Answer: Queue

Operations on a queue are performed in _____ order.

Answer: FIFO

Insertion operation in a queue is called _____ and deletion operation in a queue is called _____.

Answer: enqueue, dequeue

Deletion of elements is performed from _____ end of the queue.

Answer: front

Elements 'A','S','D' and 'F' are present in the queue, and they are deleted one at a time, _____ is the sequence of element received.

Answer: 'A', 'S', 'D', 'F'

_____ is a data structure where elements can be added or removed at either end, but not in the middle.

Answer: Deque

A deque contains 'z','x','c','v' and 'b'. Elements received after deletion are 'z','b','v','x' and 'c'. _____ is the sequence of deletion operation performed on deque.

Answer: deletionFront(), deletionRear(), deletionRear(), deletionFront(), deletionFront()

2. Compare and contrast queue with stack.

Answer: A queue operates on the First-In-First-Out (FIFO) principle, where insertion (enqueue) happens at the rear and deletion (dequeue) at the front. In contrast, a stack works on the Last-In-First-Out (LIFO) principle, with both insertion (push) and deletion (pop) at the top end. Thus, a queue is like a line, while a stack is like a pile.

3. How does FIFO describe queue?

Answer: FIFO (First-In-First-Out) means the element inserted earliest in the queue is removed first. In a queue, elements are always added at the rear and removed from the front, so the oldest element exits before newer ones, following FIFO order.

4. Write a menu driven Python program using queue, to implement movement of shuttlecock in its box.

Answer:

```
queue = []

def display_queue():
    if not queue:
        print("Queue is empty")
    else:
        print("Current queue:", queue)

def enqueue(item):
    queue.append(item)

def dequeue():
    if queue:
        removed = queue.pop(0)
        print("Removed", removed, "from the queue")
    else:
        print("Queue is empty, cannot remove")

while True:
    print("\nMENU:")
    print("1. Add movement to the queue")
    print("2. Remove movement from the queue")
    print("3. Display current queue")
    print("4. Exit")
    choice = input("Enter your choice (1-4): ")
```

```
if choice == '1':
    movement = input("Enter the movement (up/down): ")
    enqueue(movement)
    print("Added", movement, "to the queue")
elif choice == '2':
    dequeue()
elif choice == '3':
    display_queue()
elif choice == '4':
    print("Exiting program...")
    break
else:
    print("Invalid choice, please try again")
```

5. How is queue data type different from deque data type?

Answer: A queue allows insertion at one end (rear) and removal from the other end (front) only. Deque (double-ended queue) allows insertion and deletion from both ends, providing more flexibility in handling data from either end.

6. Show the status of queue after each operation:

enqueue(34) : [] → [34]

enqueue(54) : [34] → [34, 54]

dequeue() : [34, 54] → [54]

enqueue(12) : [54] → [54, 12]

dequeue() : [54, 12] → [12]

enqueue(61) : [12] → [12, 61]

peek() : [12, 61] → PEEK = 12

dequeue() : [12, 61] → [61]

dequeue() : [61] → []

dequeue() : [] → Underflow (queue empty)

dequeue() : [] → Underflow (queue empty)

enqueue(1) : [] → [1]

7. Show the status of deque after each operation:

peek() : [] → peek = Queue empty

insertFront(12) : [] → [12]

insertRear(67) : [12] → [12, 67]

deletionFront() : [12, 67] → [67]

insertRear(43) : [67] → [67, 43]

deletionRear() : [67, 43] → [67]

deletionFront() : [67] → []

deletionRear() : [] → Underflow (deque empty)

8. Write a Python program to check whether the given string is palindrome or not, using deque.

Answer:

```
def Deque():  
    return []  
  
def addRear(deque, ch):  
    deque.append(ch)  
  
def removeFront(deque):  
    if len(deque) == 0:  
        return None  
    return deque.pop(0)  
  
def removeRear(deque):  
    if len(deque) == 0:  
        return None  
    return deque.pop()  
  
def palchecker(aString):  
    chardeque = Deque()  
    for ch in aString:  
        addRear(chardeque, ch)  
    while len(chardeque) > 1:  
        first = removeFront(chardeque)  
        last = removeRear(chardeque)
```

```
    if first != last:  
        return False  
    return True
```

```
string1 = input("Enter a string: ")  
pal = palchecker(string1.lower())  
if pal:  
    print(string1, "is a palindrome.")  
else:  
    print(string1, "is not a palindrome.")
```